Question:

A company has developed an e-commerce application using AWS Lambda. Customers need to access APIs for viewing products, placing orders, and checking order status. The company wants a secure API endpoint that can handle increasing traffic without directly exposing the backend Lambda functions.Explain how Amazon API Gateway can be used in this scenario. Discuss API creation, integration, security, throttling, and monitoring**.**

1. Introduction : Amazon API Gateway is a fully managed AWS service used to **create, publish, secure, monitor, and manage APIs**. It can act as an entry point between clients and backend services such as Lambda.

2. Architecture

Customer / Mobile App

↓

API Gateway

↓

Lambda

↓

Database

3. API Creation: The company can create REST APIs or HTTP APIs in API Gateway.

Example endpoints: GET /products ,POST /orders ,GET /orders/{id}

API Gateway receives the client request and forwards it to the appropriate Lambda function.

4. Lambda Integration : Each API route can be connected to a Lambda function.

For example:

- /products → Product Lambda
- /orders → Order Lambda
- /orders/{id} → Order Status Lambda

The Lambda function processes the request and returns the response through API Gateway.

5. Security

API Gateway can be secured using: IAM authentication ,Lambda authorizers ,Amazon Cognito ,API keys and usage plans ,HTTPS encryption

This prevents unauthorized users from accessing APIs.

6. Throttling: API Gateway provides throttling to control excessive requests.

For example: 1000 requests/second

If traffic exceeds the configured limit, additional requests can be throttled instead of overwhelming the backend.

7. Monitoring:API Gateway can be integrated with **Amazon CloudWatch** to monitor:

- Number of requests
- Latency
- Errors
- 4xx responses
- 5xx responses

8. Advantages: Fully managed service **,**Automatic scaling **,**Secure API endpoints **,**Authentication support **,**Request throttling **,**Monitoring and logging

Conclusion:API Gateway provides a secure and scalable entry point for the e-commerce application. It separates clients from backend Lambda functions while providing authentication, throttling, monitoring, and API management.

2. VPC + Subnet + Routing Table + NAT – Major Scenario

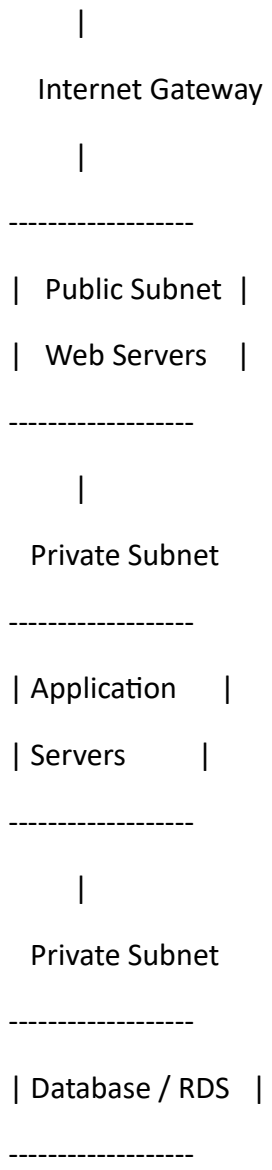
Question:A company is deploying a three-tier web application on AWS. The web servers must be publicly accessible, application servers should remain private, and database servers must not be directly accessible from the Internet.

Design an AWS VPC architecture using public/private subnets, route tables, Internet Gateway, and NAT Gateway. Explain the traffic flow.

Answer:

1. Proposed Architecture

Internet



2. VPC:A VPC provides an isolated virtual network in AWS.

Example: VPC: 10.0.0.0/16

It provides control over:

- IP addressing
- Subnets
- Routing
- Security
- Network connectivity

3. Public Subnet

A public subnet contains resources that need Internet connectivity.

Example:

10.0.1.0/24

Web servers can be placed here.

Its route table contains:

0.0.0.0/0 → Internet Gateway

4. Private Application Subnet

Example:

10.0.2.0/24

Application servers are placed here.

They should not have a direct route to the Internet Gateway.

5. Database Subnet

Example:

10.0.3.0/24

RDS/database resources can be placed in private subnets.

They should not have direct Internet access.

6. NAT Gateway

Private application servers may need Internet access for:

- Software updates
- Downloading packages
- Accessing external APIs

The private subnet can use:

0.0.0.0/0 → NAT Gateway

The NAT Gateway is placed in a public subnet.

7. Routing

Public route table:

0.0.0.0/0 → Internet Gateway

Private application route table:

0.0.0.0/0 → NAT Gateway

Database route table:

No Internet route

8. Traffic Flow

User request:

Internet

↓

Internet Gateway

↓

Web Server

↓

Application Server

↓

RDS

For outbound application traffic:

Application Server

↓

NAT Gateway

↓

Internet Gateway

↓

Internet

Conclusion

This architecture provides **security through network isolation**. Web servers are publicly accessible, application servers remain private, and databases are isolated from direct Internet access.

3. Routing Table – Higher Order Question

Question:

An application server in a private subnet cannot access the Internet, while a web server in the public subnet can. Both servers are running normally.

Analyze the possible routing problem and explain how you would troubleshoot and correct it.

Answer:

A routing table determines where network traffic should be sent.

Public Subnet

The public subnet should have:

Destination Target

10.0.0.0/16 local

0.0.0.0/0 Internet Gateway

Therefore, the web server can communicate with the Internet.

Private Subnet

The private subnet should have:

Destination Target

10.0.0.0/16 local

0.0.0.0/0 NAT Gateway

If the route is missing:

0.0.0.0/0 → NAT Gateway

the application server cannot access the Internet.

Troubleshooting Steps

1. Check the subnet associated with the application server.
2. Check which route table is associated with the subnet.

3. Verify that a default route exists.
4. Verify that the route points to a NAT Gateway.
5. Check whether the NAT Gateway is in a public subnet.
6. Verify that the NAT Gateway's subnet has an Internet Gateway route.
7. Check Network ACLs.
8. Check Security Groups.
9. Check whether the NAT Gateway is available.

Correct Architecture

Private EC2

↓

Private Route Table

↓

NAT Gateway

↓

Public Route Table

↓

Internet Gateway

↓

Internet

Conclusion

The most likely cause is an incorrect or missing default route in the private subnet. Routing tables should be checked first, followed by NAT Gateway, security groups, and network ACLs.

4. NAT Gateway – Scenario

Question:

A company wants its private EC2 instances to download software updates from the Internet but does not want Internet users to initiate connections to these EC2 instances.

Explain how NAT Gateway solves this requirement.

Answer:

NAT stands for **Network Address Translation**.

A NAT Gateway allows resources in a private subnet to initiate outbound Internet connections without allowing unsolicited inbound Internet connections to those resources.

Architecture

Private EC2

↓

Private Route Table

↓

NAT Gateway

↓

Internet Gateway

↓

Internet

Configuration

NAT Gateway should be deployed in a **public subnet**.

Public subnet route:

0.0.0.0/0 → Internet Gateway

Private subnet route:

0.0.0.0/0 → NAT Gateway

Working

Suppose an EC2 instance needs to download an update.

EC2 → NAT Gateway → Internet

The NAT Gateway performs address translation and sends the request to the Internet.

The response returns through the NAT Gateway to the EC2 instance.

Security Benefit

Internet users cannot directly initiate connections to the private EC2 instance through the NAT Gateway.

Thus:

Private EC2 → Internet Allowed

Internet → Private EC2 Not directly allowed

Conclusion

NAT Gateway provides outbound Internet connectivity to private resources while keeping those resources inaccessible from direct inbound Internet connections.

---

5. AWS KMS – Scenario-Based Question

Question:

A healthcare company stores sensitive patient information in Amazon RDS and objects in Amazon S3. The company must encrypt its data and control who can use encryption keys.

Explain how AWS Key Management Service (KMS) can be used to meet this requirement.

Answer:

1. Introduction

AWS Key Management Service (KMS) is a managed service used to **create, control, and manage cryptographic keys**.

2. Customer Managed Key

The company can create a customer managed KMS key.

The key can be used to encrypt sensitive data.

3. RDS Encryption

RDS can use a KMS key for encryption at rest.

Application

↓

RDS

↓

KMS Encryption Key

This protects database storage, backups, and snapshots.

4. S3 Encryption

S3 objects can also be encrypted using KMS.

Application

↓

S3

↓

KMS

5. Access Control

IAM policies can control who can:

- Use a key
- Encrypt data
- Decrypt data
- Create grants
- Manage keys

6. Key Rotation

KMS supports key rotation for supported key types/configurations, reducing the need to manually manage cryptographic key material.

7. CloudTrail Integration

KMS API activities can be recorded using CloudTrail.

For example:

- Who used the key?
- When was it used?
- Which API operation occurred?

8. Benefits

- Centralized key management
- Encryption at rest
- IAM integration
- Auditing
- Controlled access
- Integration with AWS services

Conclusion

KMS provides centralized and controlled encryption-key management for sensitive healthcare data stored in AWS services such as RDS and S3.

---

6. RDS vs DynamoDB – Important Scenario

Question:

A hospital wants to build two systems:

1. A patient administration system requiring relationships between patients, doctors, departments, and appointments.
2. A high-speed medication logging system receiving thousands of medication records per second.

Analyze whether Amazon RDS or DynamoDB should be used for each system.

Answer:

Requirement	**RDS**	**DynamoDB**
Database type	Relational	NoSQL
Schema	Structured	Flexible
SQL queries	Yes	No traditional SQL
Joins	Supported	Not traditional joins
Transactions	Supported	Supported

Requirement	RDS	DynamoDB
Scaling	Primarily vertical + read replicas / supported scaling options	Automatic horizontal scaling
Best for	Relational applications	High-scale key-value/document workloads

System 1: Patient Administration

RDS is suitable because healthcare administration involves relationships.

Example:

Patient

↓

Appointment

↓

Doctor

↓

Department

SQL queries can join multiple tables.

Example:

```
SELECT Patient.Name, Doctor.Name
FROM Patient
JOIN Appointment
ON Patient.ID = Appointment.PatientID
JOIN Doctor
ON Appointment.DoctorID = Doctor.ID;
```

System 2: Medication Logs

DynamoDB is suitable when medication records need very fast key-based access at high scale.

Possible key design:

Partition Key: PatientID

Sort Key: MedicationTimestamp

This allows efficient retrieval of medication records for a patient.

Conclusion

Use **RDS for relational hospital administration** and **DynamoDB for high-volume, low-latency medication logs**, provided the access patterns fit a key-value/document model.

7. CloudFormation – Higher Order Question

Question:

An organization manually creates VPCs, subnets, EC2 instances, security groups, and RDS databases for different environments. This causes configuration differences between development, testing, and production.

Explain how AWS CloudFormation can solve this problem.

Answer:

AWS CloudFormation is an Infrastructure as Code (IaC) service that allows infrastructure to be defined in templates.

Without CloudFormation

Manual Creation

↓

Development

↓

Testing

↓

Production

Configuration errors may occur.

With CloudFormation

CloudFormation Template

↓

Stack Creation

↓

VPC + Subnets + EC2 + RDS + Security Groups

Template

A CloudFormation template can define:

- VPC
- Subnets
- Route tables
- Internet Gateway
- Security Groups
- EC2
- RDS
- IAM resources

Advantages

1. Automation

Resources can be created automatically.

2. Consistency

The same template can be used for multiple environments.

3. Version Control

Templates can be stored in Git.

4. Repeatability

The same infrastructure can be recreated.

5. Dependency Management

CloudFormation determines resource dependencies.

6. Stack Management

Infrastructure can be managed as a single stack.

Example

Template

↓

Dev Stack

↓

Test Stack

↓

Production Stack

Conclusion

CloudFormation reduces manual configuration and creates consistent, repeatable, and manageable AWS infrastructure.

8. CloudTrail – Security Scenario

Question:

An administrator discovers that an important AWS resource was modified unexpectedly. The organization wants to determine who performed the action, when it occurred, and from where.

Explain how AWS CloudTrail can help investigate the incident.

Answer:

AWS CloudTrail records AWS API activity and provides an audit trail of actions performed in an AWS account.

Information Available

CloudTrail events can contain information such as:

- User identity
- API operation
- Time of operation
- AWS service

- Resource involved
- Source IP address
- Request details

Investigation Process

Unexpected Change

↓

CloudTrail

↓

Find Event

↓

Identify User/Role

↓

Check Time

↓

Check Source

↓

Investigate Further

Example

Suppose an RDS configuration was modified.

CloudTrail can help identify:

Service: RDS

Operation: ModifyDBInstance

Identity: IAM Role/User

Time: Recorded event time

Source IP: Recorded source address

Security Benefits

- Auditing
- Incident investigation
- Compliance
- User activity tracking
- Detection of unauthorized API activity

CloudTrail vs CloudWatch

CloudTrail	**CloudWatch**
Records API activity	Monitors metrics/logs/events
Security auditing	Performance monitoring
Who performed an action?	How is the system performing?

Conclusion

CloudTrail is primarily used for **API auditing and governance**, making it useful for investigating unexpected or unauthorized AWS changes.

9. CloudWatch – Performance Scenario

Question:

An application running on EC2 occasionally becomes slow. The administrator wants to automatically detect high CPU utilization and receive an alert.

Explain how Amazon CloudWatch can be configured for this requirement.

Answer:

Amazon CloudWatch is used for monitoring AWS resources, applications, metrics, and logs.

Architecture

EC2

↓

CloudWatch Metrics

↓

CloudWatch Alarm

↓

SNS Notification

Step 1: Monitor Metrics

Important EC2 metrics include:

- CPU utilization
- Network traffic
- Disk-related metrics where available/configured
- Status checks

Step 2: Create Alarm

Example:

CPU Utilization > 80%

for 5 minutes

CloudWatch continuously evaluates the metric.

Step 3: Notification

If the condition is satisfied:

CloudWatch Alarm

↓

SNS

↓

Administrator

The administrator receives an alert.

Step 4: Automatic Action

The alarm can also be associated with scaling actions where appropriate.

For example:

High CPU

↓

Alarm

↓

Auto Scaling

↓

Launch additional EC2 instance

Benefits

- Real-time monitoring
- Automated alerts
- Performance analysis
- Troubleshooting
- Integration with Auto Scaling

Conclusion

CloudWatch allows administrators to continuously monitor EC2 performance and automatically respond to abnormal conditions.

10. CI/CD – Very Important Scenario

Question:

A software company manually builds and deploys its application to EC2. Every update requires developers to manually test, build, and upload files. This causes deployment delays and errors.

Design a CI/CD pipeline using AWS services and explain its working.

Answer:

CI/CD stands for:

- **Continuous Integration**
- **Continuous Delivery/Deployment**

Proposed Pipeline

Developer

↓

CodeCommit / GitHub

↓

CodeBuild

↓

Testing

↓

CodeDeploy

↓

EC2 / ECS / Lambda

AWS also provides CodePipeline to orchestrate stages.

1. Source Stage

Developer pushes code to a source repository such as GitHub or CodeCommit.

2. Build Stage

AWS CodeBuild can:

- Compile code
- Install dependencies
- Run tests
- Create build artifacts

3. Testing

Automated tests are executed.

If tests fail:

Pipeline → STOP

If tests pass:

Pipeline → Continue

4. Deployment

AWS CodeDeploy can deploy the application to EC2.

Deployment strategies can reduce downtime and deployment risk, depending on architecture.

5. Monitoring

CloudWatch can monitor application and infrastructure performance.

Advantages

- Faster releases
- Automated testing
- Reduced manual errors
- Consistent deployments
- Faster feedback
- Better reliability

Conclusion

A CI/CD pipeline automates the software lifecycle from code commit to testing and deployment, reducing manual work and improving release consistency.

11. CloudWatch + Auto Scaling – Higher Order

Question:

An e-commerce website experiences normal traffic during the day but extremely high traffic during sales. During high traffic, the existing EC2 instances become overloaded.

Explain how CloudWatch and Auto Scaling can be combined to automatically handle this situation.

Answer:

Architecture

Users

↓

ALB

↓

Auto Scaling Group

↓

EC2 Instances

↓

CloudWatch

Working

CloudWatch continuously monitors metrics such as CPU utilization.

Example condition:

Average CPU > 70%

CloudWatch alarm triggers the scaling policy.

High CPU

↓

CloudWatch Alarm

↓

Auto Scaling

↓

Launch EC2

When demand decreases:

Low CPU

↓

CloudWatch Alarm

↓

Auto Scaling

↓

Remove unnecessary instances

Benefits

- Handles traffic spikes

- Maintains application availability
- Reduces manual intervention
- Helps optimize resource usage
- Automatically responds to demand

Conclusion

CloudWatch provides monitoring and alarms, while Auto Scaling changes the number of EC2 instances based on demand.

12. Integrated Scenario – Very High Probability

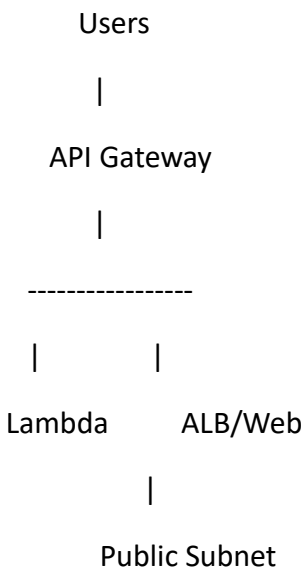
Question:

A company wants to deploy a highly available online shopping application on AWS. Customers access the application through APIs. The application requires a database, secure private servers, encryption, monitoring, auditing, and automated deployment.

Design a complete AWS architecture using API Gateway, VPC, subnets, routing tables, NAT, RDS/DynamoDB, KMS, CloudFormation, CloudTrail, CloudWatch, and CI/CD. Explain the role of each service.

Answer:

Proposed Architecture



|

Private Subnet

Application EC2

|

Private Subnet

RDS

|

KMS

Private EC2

|

NAT Gateway

|

Internet Gateway

|

Internet

1. API Gateway

Acts as the API entry point for customers.

It provides:

- API management
- Authentication
- Throttling
- Monitoring

2. VPC

Provides an isolated network for application resources.

Example:

10.0.0.0/16

3. Subnets

Public Subnet → Web/Load Balancer

Private Subnet → Application Servers

Private DB Subnet → RDS

4. Routing Tables

Public:

0.0.0.0/0 → IGW

Private:

0.0.0.0/0 → NAT Gateway

5. NAT Gateway

Allows private application servers to access the Internet for outbound requirements without directly exposing them to inbound Internet connections.

6. RDS / DynamoDB

RDS can be used for transactional relational data such as:

- Customers
- Orders
- Products

DynamoDB can be used for high-scale key-value/document workloads such as:

- Shopping sessions
- Certain event/log data
- High-volume application access patterns

7. KMS

KMS manages encryption keys for supported AWS services and helps protect sensitive data.

8. CloudFormation

Defines the infrastructure as code.

Template

↓

VPC

↓

Subnets

↓

Security Groups

↓

EC2

↓

RDS

9. CloudTrail

Records AWS API activity for auditing and security investigation.

10. CloudWatch

Monitors:

- CPU
- Network
- Application metrics
- Logs
- Alarms

11. CI/CD

A pipeline can be created using:

GitHub/CodeCommit

↓

CodePipeline

↓

CodeBuild

↓

Testing

↓

CodeDeploy

↓

Application

12. Overall Benefits

The architecture provides:

- Security
- Scalability
- Monitoring
- Encryption
- Auditing
- Automation
- High availability
- Reduced manual configuration

Conclusion

Combining these AWS services creates a secure, scalable, monitored, and automated application architecture. Each service performs a specific role while working together as a complete cloud solution.

Q.Infrastructure as Code (IaC) & Cl/ CD Pipelines Scenario: A software company is developing a cloud-hosted web application and wants to automate resource provisioning and continuous delivery. Part A: Compare AWS CloudFormation and the AWS Cloud Development Kit (CDK) in terms of developer experience, syntax, and maintainability. Part B: Explain how AWS CodePipeline integrates with CloudFormation/CDK to establish an automated Cl/CD deployment pipeline.

Introduction

Infrastructure as Code (IaC) allows developers to define and manage cloud infrastructure using code instead of creating resources manually. AWS provides **CloudFormation** and **AWS CDK** for IaC, while **AWS CodePipeline** can automate the process of building, testing, and deploying applications.

Part A: CloudFormation vs. AWS CDK

Feature	**AWS CloudFormation**	**AWS CDK**
Type	IaC service	IaC development framework
Syntax	YAML/JSON templates	Programming languages
Languages	YAML/JSON	Java, Python, TypeScript, C#, etc.
Developer experienc	More template-oriented	More programming-oriented
Reusability	Possible through templates	Easy using classes and constructs
Abstraction	Lower-level	Higher-level constructs available
Maintainability	Can become lengthy for large templ	Easier to organize complex infrastructu
Deployment	Directly creates CloudFormation sta	CDK synthesizes CloudFormation temp

AWS CloudFormation

CloudFormation allows developers to describe AWS resources in **YAML or JSON templates**.

For example, a template can define:

- EC2 instances
- VPCs
- S3 buckets
- RDS databases
- IAM roles

CloudFormation then creates and manages these resources as a **stack**.

Advantages:

- Declarative infrastructure definition.

- Direct AWS infrastructure management.
- Supports repeatable deployments.
- Useful when teams prefer YAML/JSON templates.

AWS CDK

AWS CDK allows developers to define infrastructure using familiar programming languages such as **Java, Python, TypeScript, and C#**.

For example, instead of writing a large YAML template, a developer can use programming constructs to create a VPC, subnet, or database.

The CDK application is converted into a **CloudFormation template** using the cdk synth process and can then be deployed.

Advantages:

- Uses familiar programming languages.
- Supports variables, loops, functions, and classes.
- Provides reusable **constructs**.
- Useful for large and complex infrastructure projects.

Part B: CodePipeline with CloudFormation/CDK

AWS CodePipeline is a continuous delivery service that automates the movement of code through different stages such as source, build, test, and deployment.

CI/CD Pipeline

Developer

↓

GitHub / CodeCommit

↓

AWS CodePipeline

↓

Build & Test

↓

CloudFormation / CDK

↓

AWS Infrastructure

↓

Application Deployment

Step 1: Source Stage

Developers push application and infrastructure code to a source repository such as GitHub.

CodePipeline detects the change and starts the pipeline.

Step 2: Build and Test Stage

AWS CodeBuild can:

- Compile the application.
- Run automated tests.
- Install CDK dependencies.
- Generate deployment artifacts.

Step 3: Infrastructure Deployment

If **CloudFormation** is used, CodePipeline can deploy the CloudFormation template as a stack.

If **CDK** is used:

CDK Source Code

↓

cdk synth

↓

CloudFormation Template

↓

CloudFormation

↓

AWS Resources

The pipeline can therefore use CDK to define infrastructure while CloudFormation performs the actual stack deployment.

Step 4: Application Deployment

After infrastructure is successfully deployed, the pipeline can deploy the application to services such as **Amazon EC2, ECS, Lambda, or S3**.

Benefits of the Automated Pipeline

- **Automation:** Reduces manual deployment work.
- **Consistency:** Infrastructure is deployed in a repeatable way.
- **Fast delivery:** Changes can move from source code to deployment automatically.
- **Testing:** Automated tests can be executed before deployment.
- **Version control:** Infrastructure and application code can be tracked in Git.
- **Rollback:** CloudFormation can help manage infrastructure changes and stack updates.

Conclusion

CloudFormation provides a template-based approach to IaC, while **AWS CDK** provides a programming-language-based approach and ultimately generates CloudFormation templates. **AWS CodePipeline** connects source control, testing, and deployment stages to create an automated CI/CD workflow. Together, these services provide a repeatable and automated method for deploying both cloud infrastructure and applications.

Q.High Availability, Auto Scaling, and Load Balancing for High-Traffic Applications Scenario: An e-commerce platform hosted on Amazon EC2 experiences extreme traffic surges during seasonal festival sales. Part A: Design a scalable architecture using Application Load Balancers (ALB) and Auto Scaling Groups (ASG) to handle sudden spikes in demand. Part B: Describe how path-based routing rules and target groups can distribute traffic dynamically across different backend server clusters during peak traffic.

Introduction

During seasonal festival sales, an e-commerce website may receive a sudden increase in users and requests. To handle this traffic without affecting performance, AWS can use **Application Load**

Balancer (ALB) with **Auto Scaling Groups (ASG)**. This provides scalability, high availability, and efficient distribution of traffic.

Part A: Scalable Architecture using ALB and ASG

1. Application Load Balancer

The **ALB** acts as the entry point for users. It receives HTTP/HTTPS requests and distributes them among healthy EC2 instances.

It can be deployed across multiple **Availability Zones (AZs)** to improve availability.

2. Auto Scaling Group

An **Auto Scaling Group** manages the EC2 instances running the application.

For example:

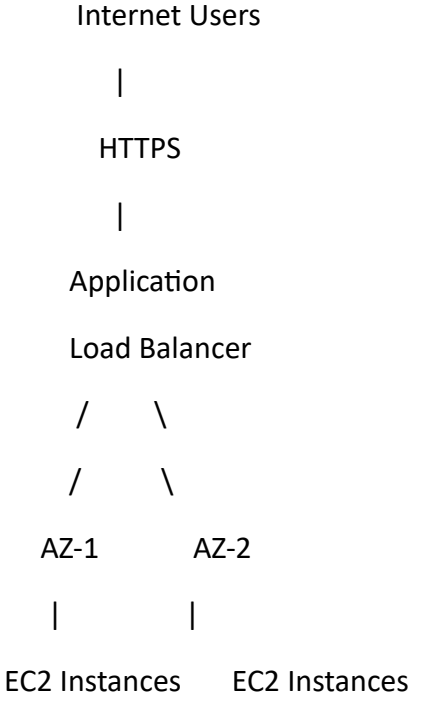
Normal Traffic → 2 EC2 Instances

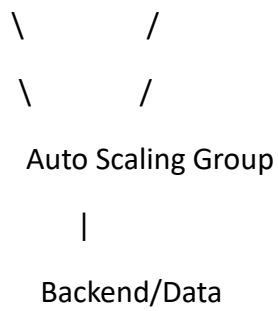
High Traffic → 5 EC2 Instances

Extreme Traffic → 10 EC2 Instances

When traffic increases, ASG launches additional EC2 instances. When traffic decreases, unnecessary instances can be terminated.

3. Proposed Architecture





4. Scaling Policies

Auto Scaling can use metrics such as:

- CPU utilization
- Number of requests
- Request count per target
- Application performance metrics

For example, if average CPU utilization increases above a defined threshold, the ASG can automatically launch additional EC2 instances.

Benefits

- Handles sudden traffic spikes.
- Maintains application availability.
- Automatically adds/removes EC2 instances.
- ALB distributes traffic across healthy instances.
- Multiple Availability Zones reduce dependence on a single AZ.

Part B: Path-Based Routing and Target Groups

1. Target Groups

A **target group** is a collection of backend targets, such as EC2 instances. An ALB forwards requests to the appropriate target group.

For example:

Target Group 1 → Product Servers

Target Group 2 → Order Servers

Target Group 3 → Payment Servers

Each target group can have its own Auto Scaling Group.

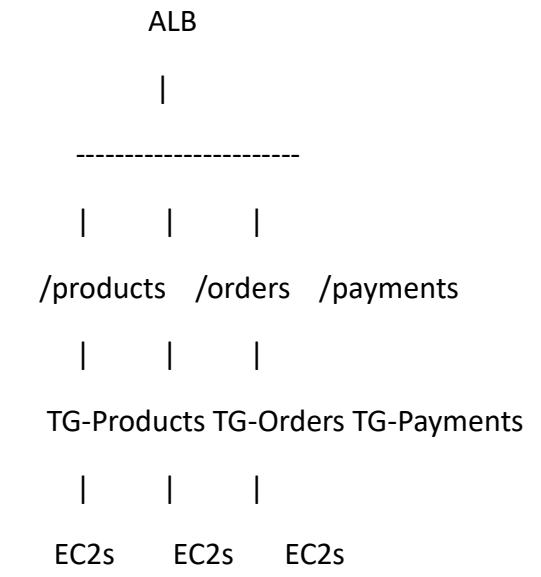
2. Path-Based Routing

ALB can examine the **URL path** and forward the request to the appropriate target group.

Example:

URL Path	Target Group	Backend
/products/*	TG-Products	Product servers
/orders/*	TG-Orders	Order servers
/payments/*	TG-Payments	Payment servers

The routing process can be represented as:



3. Dynamic Distribution During Peak Traffic

During a festival sale, the number of requests to different parts of the application may vary.

For example, if product browsing receives very high traffic:

/products

↓

TG-Products

↓

ASG automatically adds EC2 instances

The ALB then distributes requests among the available healthy instances in that target group.

Similarly, if order processing experiences increased traffic, the **Order ASG** can scale independently.

4. Health Checks

ALB performs health checks on targets. If an EC2 instance becomes unhealthy, ALB stops sending new requests to that instance and routes traffic to healthy targets.

Conclusion

An architecture using **ALB + ASG + multiple Availability Zones** can handle sudden traffic increases while maintaining high availability. **Path-based routing** directs different types of requests to appropriate target groups, while independent Auto Scaling Groups allow each backend cluster to scale according to its workload. This makes the e-commerce application more flexible and resilient during peak festival traffic.

Q.Infrastructure Performance & Health Monitoring Scenario: To ensure high reliability, an system administrator needs to monitor system performance and server availability across network layers. Part A: Explain the difference between Layer 3 (Network Level) and Layer 7 (Application Level) health checks and performance monitoring on AWS. Part B: Outline how CloudWatch metrics and ALB health checks are configured to automatically replace underperforming or degraded EC2 instances.

Introduction

Monitoring is important for maintaining the **availability, performance, and reliability** of cloud applications. AWS provides monitoring tools such as **Amazon CloudWatch** and **Application Load Balancer (ALB) health checks** to detect unhealthy or underperforming resources.

Part A: Layer 3 vs. Layer 7 Monitoring

1. Layer 3 — Network Level

Layer 3 operates at the **Network Layer** and mainly deals with IP connectivity and routing.

Network-level monitoring checks whether a resource is reachable over the network.

Examples:

- IP connectivity
- Network traffic
- Packet loss
- Latency
- Network throughput
- Route availability

AWS services and features such as **VPC Flow Logs** and CloudWatch network metrics can help monitor network activity.

Example:
If an EC2 instance is not reachable through its network path, a Layer 3 problem may exist.

2. Layer 7 — Application Level

Layer 7 operates at the **Application Layer** and checks whether the actual application is working correctly.

ALB health checks commonly use HTTP/HTTPS requests to test an application endpoint.

Example:

ALB → HTTP GET /health

↓

EC2 Application

↓

HTTP 200 = Healthy

If the application is running but returning errors such as **HTTP 500**, the instance may be considered unhealthy even though its network connection is working.

Difference

Feature	**Layer 3**	**Layer 7**
Layer	Network	Application

Feature	Layer 3	Layer 7
Checks	Network connectivity	Application functionality
Examples	IP, latency, packet loss	HTTP/HTTPS response
Main purpose	Network availability	Application availability
AWS example	VPC Flow Logs, network metrics	ALB health checks

Part B: CloudWatch Metrics and ALB Health Checks

1. CloudWatch Monitoring

Amazon CloudWatch collects metrics from AWS resources such as EC2 and ALB.

Important EC2 metrics include:

- CPU utilization
- Network traffic
- Disk-related metrics where available
- Status checks

ALB metrics can include:

- Request count
- Target response time
- HTTP error responses
- Healthy/unhealthy target counts

2. ALB Health Checks

A target group can be configured with:

- **Protocol:** HTTP/HTTPS
- **Path:** /health
- **Port:** Application port
- **Interval:** Time between checks

- **Timeout:** Time allowed for response
- **Healthy threshold:** Number of successful checks
- **Unhealthy threshold:** Number of failed checks

Example:

ALB

|

|--- Health Check: GET /health

|

+---- EC2-1 → Healthy

|

+---- EC2-2 → Unhealthy

|

+---- EC2-3 → Healthy

The ALB stops routing new requests to an unhealthy target.

3. Automatic Replacement using ASG

For automatic replacement, the EC2 instances should be managed by an **Auto Scaling Group (ASG)**.

The process is:

EC2 Instance

↓

CloudWatch / ALB detects problem

↓

Instance becomes unhealthy

↓

ALB stops sending traffic

↓

ASG detects unhealthy instance

↓

Terminates/replaces instance

↓

New EC2 instance launched

↓

ALB health check passes

↓

Instance receives traffic

4. CloudWatch Alarms

CloudWatch alarms can monitor selected metrics and trigger actions when thresholds are crossed.

For example:

CPU Utilization > 80% for several minutes

→ CloudWatch alarm
→ ASG scaling policy
→ Launch additional EC2 instance

Similarly, an unhealthy-instance condition can lead to replacement when the ASG's health-check configuration detects the instance as unhealthy.

Conclusion

Layer 3 monitoring focuses on network connectivity and performance, while **Layer 7 monitoring** verifies whether the application itself is responding correctly. CloudWatch provides infrastructure and application-related metrics, while ALB health checks identify unhealthy application targets. When combined with an **Auto Scaling Group**, unhealthy instances can be removed and replaced automatically, improving application availability and reliability.

Q.Two teams propose different architectures for an online shopping application:

Multiple EC2 instances but no load balancing.

Multiple EC2 instances with load balancing and automatic scaling.

The application experiences unpredictable traffic throughout the day.

Ans

For an online shopping application with unpredictable traffic, the architecture with **multiple EC2 instances, load balancing, and automatic scaling** is more suitable because it can distribute traffic and dynamically adjust resources according to demand.

1. Architecture A – Multiple EC2 Instances Without Load Balancing

In this architecture, multiple EC2 instances are available, but there is no load balancer.

Users

| | |

↓ ↓ ↓

EC2 1 EC2 2 EC2 3

Users or the application must somehow decide which instance to access.

Problems:

- Traffic may not be distributed evenly.
- One instance may become overloaded while others remain underutilized.
- There is no automatic replacement or scaling based on traffic.
- A failed instance can affect users if they are directly connected to it.
- Managing changing traffic becomes difficult.

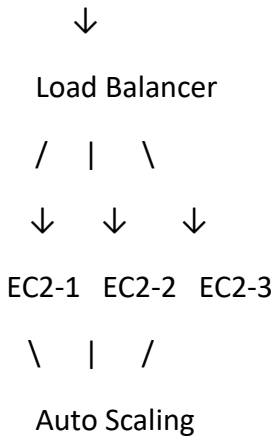
Therefore, simply adding multiple EC2 instances does not automatically provide efficient traffic distribution.

2. Architecture B – Load Balancer + Auto Scaling

The second architecture introduces an **Application Load Balancer (ALB)** and an **Auto Scaling Group**.

Users

|



The Load Balancer distributes incoming requests across healthy EC2 instances.

3. Load Balancing

The load balancer receives requests from customers and distributes them among available EC2 instances.

For example:

Request 1 → EC2-1

Request 2 → EC2-2

Request 3 → EC2-3

Request 4 → EC2-1

This prevents a single instance from receiving all the traffic.

The load balancer can also perform **health checks** and avoid sending traffic to unhealthy instances.

4. Automatic Scaling

An Auto Scaling Group can automatically increase or decrease the number of EC2 instances.

During high traffic:



↓

Auto Scaling

↓

Launch more EC2 instances

During low traffic:

Low Traffic

↓

Auto Scaling

↓

Terminate unnecessary instances

5. Role of CloudWatch

Amazon CloudWatch can monitor metrics such as CPU utilization and other application or load-balancer metrics.

For example:

CPU > 70%

↓

CloudWatch Alarm

↓

Auto Scaling

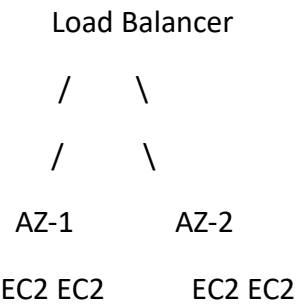
↓

Add EC2 instances

This allows the architecture to respond automatically to unpredictable traffic.

6. High Availability

EC2 instances can be distributed across multiple **Availability Zones**.



If an instance or Availability Zone has a problem, traffic can be directed to healthy instances.

7. Comparison

Feature	**Multiple EC2 Only**	**EC2 + Load Balancer + Auto Scaling**
Traffic distribution	Limited/manual	Automatic
Scaling	Manual	Automatic
Health checks	Limited	Supported through LB
Handling traffic spikes	Difficult	Better
Fault tolerance	Lower	Higher
Resource utilization	May be inefficient	More dynamic
Suitable for unpredictable traffic	Less suitable	Suitable

Conclusion

For an online shopping application with unpredictable traffic, **Architecture B provides load distribution and automatic scaling**. The combination of **ALB + Auto Scaling + CloudWatch + multiple EC2 instances across Availability Zones** allows the application to handle changing traffic while improving availability and resource utilization.

A healthcare organization stores confidential patient-related files in AWS. An administrator discovers that several unauthorized users have gained access to encrypted data because they were given permissions associated with the encryption key.

Ans

The problem is related to **improper access control for the AWS KMS encryption key**. Encryption alone does not guarantee that data is inaccessible to unauthorized users. Users who have appropriate permissions to use a KMS key may be able to decrypt protected data.

1. Role of AWS KMS

AWS Key Management Service (KMS) is used to create and control cryptographic keys used to protect data.

For example:

Patient Files

↓

Encrypted using KMS Key

↓

Encrypted Data

Only appropriately authorized users or AWS services should be allowed to use the key for decryption.

2. Identify the Problem

The administrator should examine:

- IAM policies
- KMS key policies
- IAM roles
- Grants
- Users/groups with kms:Decrypt permission

The main concern is excessive permissions such as:

kms:Decrypt

being granted to users who do not need to decrypt patient data.

3. Review IAM Permissions

The organization should follow the **principle of least privilege**.

Instead of:

All Employees

↓

kms:Decrypt

permissions should be limited to the required users, roles, or services.

For example:

Authorized Application Role

↓

KMS Key

↓

Decrypt

Users who only need to upload files may not require decryption permission.

4. Review KMS Key Policy

The KMS key policy determines who can use and manage the key.

The administrator should remove unnecessary principals and restrict permissions such as:

- kms:Decrypt
- kms:Encrypt
- kms:GenerateDataKey
- Key-management permissions

The key policy should allow access only to trusted roles and services.

5. Separate Key Administration and Key Usage

The organization should avoid giving the same users both:

- Key administration permissions
- Data decryption permissions

For example:

Security Administrator

↓

Manage KMS Key

Application Role

↓

Use KMS Key for required operations

This separation reduces the possibility of unauthorized access.

6. Use IAM Roles Instead of Broad User Permissions

Applications should preferably use appropriate IAM roles rather than giving individual users broad permissions.

For example:

Application

↓

IAM Role

↓

KMS

↓

Decrypt required data

The role should contain only the permissions required by the application.

7. Audit KMS Activity Using CloudTrail

AWS CloudTrail can be used to investigate KMS API activity.

The organization can examine events related to operations such as:

kms:Decrypt

kms:Encrypt

kms:GenerateDataKey

This can help determine:

- Which identity accessed the key
- When the key was accessed
- Which API operation was performed
- Source information associated with the request

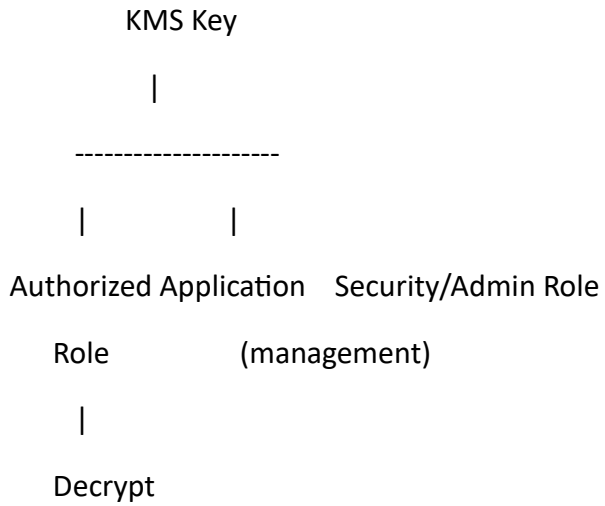
8. Remove Unauthorized Access

After identifying the unauthorized users, the administrator should:

1. Remove unnecessary KMS permissions.
2. Update IAM policies.
3. Review the KMS key policy.
4. Review KMS grants where applicable.
5. Restrict access to approved roles.
6. Review CloudTrail logs for suspicious activity.
7. Continue monitoring future key usage.

9. Principle of Least Privilege

The key should be accessible only to identities that actually require it.



Unnecessary users should not have access.

10. Important Point

Encryption does not replace authorization.

Even though the files are encrypted, a user with sufficient permission to use the KMS key may be able to decrypt the data.

Therefore, security requires both:

Encryption + Proper Access Control

Conclusion

The healthcare organization should immediately review **IAM policies, KMS key policies, roles, and grants**, remove unnecessary kms:Decrypt permissions, and follow the principle of least privilege. CloudTrail should be used to audit previous key usage and investigate unauthorized access. This approach provides stronger protection for confidential patient information.